UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL

CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO– JULIO 2026

I. PORTADA

Tema: Evaluacion UNIDAD 2

Unidad de Organización Curricular: BÁSICA

Nivel y Paralelo: 1ro Software B

Alumnos participantes: Rocha Rocha Carolina Estefania

Segovia Garcia Joseph Andre

Barrionuevo Montesdeoca Job Gabriel

Asignatura: Algoritmos y Lógica de programación

Docente: Ing. Jose Ruben Caizabuano, Mg.

Tema: Mantenimiento técnico de equipos UTA

II. INFORME DE EVALUACIÓN

2.1 Objetivos

General:

Desarrollar un sistema en Visual Studio Code - C + + que permita gestionar órdenes de mantenimiento técnico de equipos, integrando un módulo de Ahorcado, aplicando buenas prácticas de programación, validación de datos y el uso de herramientas de control de versiones. El proyecto será desarrollado en Visual Studio Code, garantizando una correcta organización del código, compilación y ejecución del sistema.

Específicos:

- Analizar el problema y definir entradas, procesos y salidas del sistema.
- Diseñar clases y estructuras necesarias para la solución.
- Implementar funciones para registro, búsqueda, ordenamiento y reportes.
- Integrar un módulo de Ahorcado relacionado con el dominio del sistema.
- Aplicar validaciones para evitar errores en la ejecución.
- Utilizar GitHub para evidenciar el proceso de desarrollo

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales:

No presenciales: 0

2.4 Instrucciones

2.5 Listado de equipos, materiales y recursos

Inteligencia artificial, TAC
- Computador o laptop.
- Plataforma educativa de la UTA.
- Internet para consultas.
- Visual Studio Code
- GitHub

.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☒Simuladores y laboratorios virtuales
☒Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación

☒Inteligencia Artificial
Otros (Especifique): Lenguaje de programación C++, IDE para programar (VisualStudio Code, CodeBlocks)

2.6 Actividades por desarrollar

- Analice la información de el ejercicio propuesto
- Realice el diseño, codificación y posteriormente compilar, ejecutar y verificar el resultado.
- Subir a la plataforma el/los archivo(s) solicitados hasta la fecha indicada

2.7. Desarrollo / Anexos
Ejercicio .

1) **<u>ENUNCIADO</u>**

Mantenimiento técnico de equipos UTA

Contexto del caso: Registrar fallas, diagnósticos, técnicos y costos de mantenimiento preventivo o correctivo de equipos institucionales.

Clases mínimas sugeridas: Mantenimiento y Técnico

Requerimientos funcionales obligatorios

- Registrar órdenes con código, equipo, fechas, tipo, técnico, costo estimado y estado.
- Permitir cambios de estado y validar consistencia entre fechas de reporte, inicio y cierre.
- Recalcular costo final con repuestos y mano de obra.
- Buscar por técnico y código de equipo, ordenar manualmente por mayor costo y mayor tiempo de resolución.
- Reportar equipos con mayor gasto, órdenes críticas y recomendaciones de reemplazo.

Integración obligatoria con Ahorcado: Construir un Ahorcado con fallas, diagnósticos, repuestos, herramientas y estados de mantenimiento.

Fragmento defectuoso para depurar y justificar

```
int diasProceso(int inicio, int cierre){
        int dias;
        if(cierre < inicio)
        dias = inicio - cierre;
        else
        dias = cierre + inicio;
        return dias;
 }
```

2) ANÁLISIS DEL PROBLEMA

El presente problema consiste en desarrollar un sistema informático en Visual Studio Code - C + + que permita gestionar de manera eficiente el mantenimiento técnico de equipos pertenecientes a una institución. Este sistema tiene como finalidad organizar y controlar la información relacionada con fallas, diagnósticos, técnicos responsables y costos asociados a los procesos de mantenimiento.

El mantenimiento de equipos puede generar desorden si no se lleva un registro adecuado, lo que dificulta el seguimiento de órdenes, el control de gastos y la identificación de equipos que requieren reemplazo. Por esta razón, el sistema propuesto busca centralizar la información y facilitar su gestión mediante herramientas de registro, consulta y análisis.

El sistema deberá permitir el registro de órdenes de mantenimiento, donde cada orden contiene información relevante como un código único, el equipo intervenido, las fechas del proceso (reporte, inicio y cierre), el tipo de mantenimiento (preventivo o correctivo), el técnico asignado, el costo estimado y el estado actual de la orden. Esta información permitirá dar seguimiento completo a cada intervención realizada.

Además, el sistema debe garantizar la consistencia de los datos mediante validaciones, especialmente en lo relacionado con las fechas, evitando incoherencias como fechas de cierre anteriores a la fecha de inicio. También se deben validar los costos para evitar valores negativos y asegurar que el estado de la orden sea coherente con su progreso.

Otro aspecto importante del sistema es la capacidad de recalcular el costo final del mantenimiento, considerando tanto los repuestos utilizados como la mano de obra, lo cual permite obtener un valor real del gasto asociado a cada orden.

El sistema también debe ofrecer funcionalidades de consulta, permitiendo buscar órdenes de mantenimiento por técnico o por código de equipo, facilitando el acceso rápido a la información. Asimismo, debe implementar mecanismos de ordenamiento manual para organizar las órdenes según criterios relevantes, como el mayor costo o el mayor tiempo de resolución, lo cual es útil para identificar casos críticos.

En cuanto a los reportes, el sistema debe ser capaz de generar información relevante como los equipos con mayor gasto en mantenimiento, órdenes consideradas críticas debido a su alto costo o duración, y recomendaciones sobre posibles reemplazos de equipos que presentan fallas frecuentes o costos elevados.

Adicionalmente, el sistema debe integrar un módulo del juego Ahorcado, el cual no debe funcionar como un elemento aislado, sino como una extensión del sistema principal. Este módulo utilizará palabras relacionadas con el dominio del mantenimiento técnico, como fallas, diagnósticos, repuestos, herramientas y estados, reforzando la coherencia temática del proyecto.

El módulo Ahorcado debe permitir la interacción del usuario mediante el ingreso de letras, validando que sean caracteres correctos, evitando repeticiones y mostrando el progreso de la palabra, los intentos restantes y el resultado de la partida. Además, debe incluir funcionalidades adicionales como niveles de dificultad, historial de partidas y sistema de puntuación.

Finalmente, el sistema debe cumplir con restricciones técnicas específicas, como el uso de programación orientada a objetos, evitando variables globales, implementando búsqueda y ordenamiento manual, y utilizando al menos una función recursiva con una utilidad real dentro del sistema.

En conclusión, el problema plantea el desarrollo de una solución completa que no solo permita registrar información, sino también analizarla, validarla y presentarla de manera útil, integrando además un componente interactivo que complemente la experiencia del usuario.

3) DECLARACIÓN DE VARIABLES

Datos de entrada:

Código de la orden
Nombre del equipo
Tipo de mantenimiento (preventivo o correctivo)
Nombre del técnico
Estado de la orden (pendiente, en proceso, finalizado)
Fecha de reporte
Fecha de inicio
Fecha de cierre
Costo estimado
Costo de repuestos
Costo de mano de obra
Descripción de la falla
Diagnóstico del técnico

Proceso:

Validación de datos ingresados
Verificación de coherencia entre fechas
Cálculo de días de duración del mantenimiento
Cálculo del costo total (repuestos + mano de obra)
Comparación entre costo estimado y costo real
Evaluación del estado de la orden
Clasificación de órdenes críticas (alto costo o tiempo)
Búsqueda por técnico
Búsqueda por código de equipo
Ordenamiento manual por mayor costo
Ordenamiento manual por mayor tiempo de resolución

Datos de salida:

Listado de órdenes registradas
Detalle de cada mantenimiento
Costo total por orden
Días de duración del mantenimiento
Órdenes con mayor costo
Órdenes con mayor tiempo de resolución
Equipos con mayor gasto acumulado
Órdenes críticas detectadas
Recomendaciones de reemplazo de equipos
Mensajes del sistema (éxito o error)

4) ALGORITMO PASO A PASO

1. Inicio
2. Declarar variables necesarias y estructuras para almacenar múltiples órdenes de mantenimiento.
3. Inicializar el contador de registros en cero.
4. Mostrar el menú principal con las siguientes opciones:
   - 4.1. Registrar orden de mantenimiento
   - 4.2. Buscar orden
   - 4.3. Ordenar órdenes
   - 4.4. Mostrar reportes
   - 4.5. Jugar Ahorcado
   - 4.6. Salir
5. Leer la opción ingresada por el usuario.
6. Según la opción seleccionada:
   OPCIÓN 1: Registrar orden
   - 6.1. Solicitar código de la orden
   - 6.2. Solicitar nombre del equipo

6.3. Solicitar tipo de mantenimiento
6.4. Solicitar nombre del técnico
6.5. Solicitar estado
6.6. Solicitar fecha de inicio
6.7. Solicitar fecha de cierre
6.8. Solicitar costo estimado
6.9. Solicitar costo de repuestos
6.10. Solicitar costo de mano de obra
6.11. Validar que los datos sean correctos
6.12. Calcular el costo total (repuestos + mano de obra)
6.13. Calcular los días de proceso (fecha cierre - fecha inicio)
6.14. Guardar los datos en los arreglos
6.15. Incrementar el contador
6.16. Mostrar mensaje de registro exitoso

7. OPCIÓN 2: Buscar orden
   7.1. Solicitar criterio de búsqueda (técnico o código)
   7.2. Recorrer los registros
   7.3. Comparar datos
   7.4. Si encuentra coincidencia, mostrar la información
   7.5. Si no encuentra, mostrar mensaje de error
8. OPCIÓN 3: Ordenar órdenes
   8.1. Elegir criterio:
      a) Por mayor costo
      b) Por mayor tiempo de resolución
   8.2. Aplicar ordenamiento manual (burbuja o similar)
   8.3. Mostrar lista ordenada
9. OPCIÓN 4: Reportes
   9.1. Mostrar todas las órdenes registradas
   9.2. Identificar la orden con mayor costo
   9.3. Identificar la orden con mayor tiempo
   9.4. Mostrar equipos con mayor gasto
   9.5. Mostrar órdenes críticas
10. OPCIÓN 5: Ahorcado
   10.1. Seleccionar palabra aleatoria relacionada con mantenimiento
   10.2. Inicializar palabra oculta
   10.3. Inicializar número de intentos
   10.4. Mientras no se adivine la palabra y haya intentos:
      10.4.1. Mostrar estado actual
      10.4.2. Solicitar letra
      10.4.3. Validar letra
      10.4.4. Verificar si la letra está en la palabra
      10.4.5. Actualizar palabra oculta

10.4.6. Restar intentos si falla
10.5. Mostrar resultado (ganó o perdió)
11. OPCIÓN 6: Salir
11.1. Mostrar mensaje de salida
12. Repetir el menú hasta que el usuario seleccione salir.
13. Fin

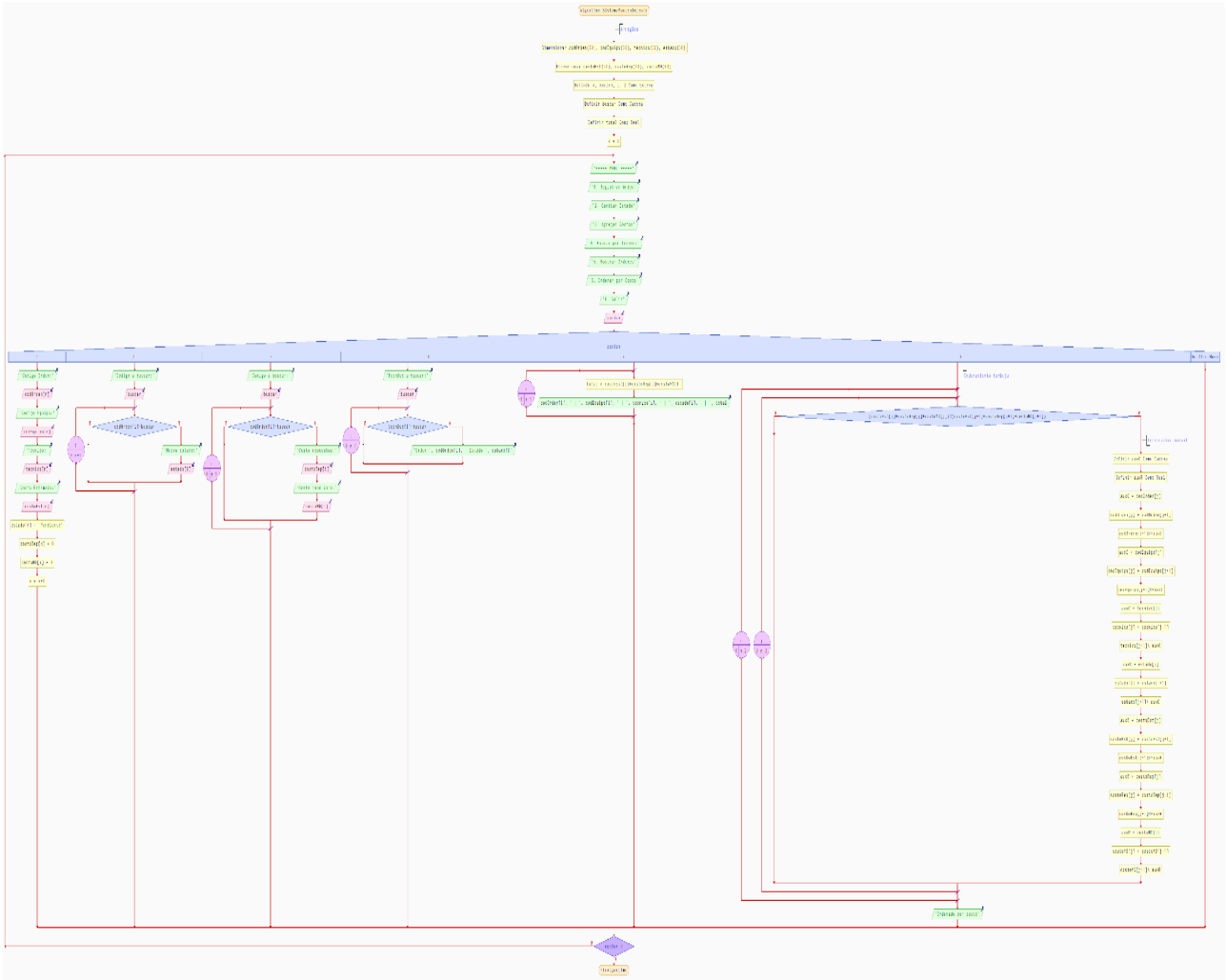
5) <u>PSEUDOCODIGO EN PSEINT</u>

```
Algoritmo SistemaMantenimiento
    // Arreglos
    Dimensionar codOrden(50), codEquipo(50), tecnico(50), estado(50)
    Dimensionar costoEst(50), costoRep(50), costoMO(50)
    Definir n, opcion, i, j Como Entero
    Definir buscar Como Cadena
    Definir total Como Real
    n <- 0
    Repetir
        Escribir '----- MENU -----'
        Escribir '1. Registrar Orden'
        Escribir '2. Cambiar Estado'
        Escribir '3. Agregar Costos'
        Escribir '4. Buscar por Tecnico'
        Escribir '5. Mostrar Ordenes'
        Escribir '6. Ordenar por Costo'
        Escribir '0. Salir'
        Leer opcion
        Según opcion Hacer
            1:
                Escribir 'Codigo Orden:'
                Leer codOrden[n]
                Escribir 'Codigo Equipo:'
                Leer codEquipo[n]
                Escribir 'Tecnico:'
                Leer tecnico[n]
                Escribir 'Costo Estimado:'
                Leer costoEst[n]
                estado[n] <- 'Pendiente'
                costoRep[n] <- 0
                costoMO[n] <- 0
                n <- n+1
            2:
                Escribir 'Codigo a buscar:'
                Leer buscar
```

```
        Para i<-0 Hasta n-1 Hacer
            Si codOrden[i]=buscar Entonces
                Escribir 'Nuevo estado:'
                Leer estado[i]
            FinSi
        FinPara
    3:
        Escribir 'Codigo a buscar:'
        Leer buscar
        Para i<-0 Hasta n-1 Hacer
            Si codOrden[i]=buscar Entonces
                Escribir 'Costo repuestos:'
                Leer costoRep[i]
                Escribir 'Costo mano obra:'
                Leer costoMO[i]
            FinSi
        FinPara
    4:
        Escribir 'Tecnico a buscar:'
        Leer buscar
        Para i<-0 Hasta n-1 Hacer
            Si tecnico[i]=buscar Entonces
                Escribir 'Orden:', codOrden[i], ' Estado:', estado[i]
            FinSi
        FinPara
    5:
        Para i<-0 Hasta n-1 Hacer
            total <- costoEst[i]+costoRep[i]+costoMO[i]
            Escribir codOrden[i], ' | ', codEquipo[i], ' | ', tecnico[i], ' | ', estado[i], ' | ',
total
        FinPara
    6:
        // Ordenamiento burbuja
        Para i<-0 Hasta n-2 Hacer
            Para j<-0 Hasta n-2 Hacer
                Si
(costoEst[j]+costoRep[j]+costoMO[j])<(costoEst[j+1]+costoRep[j+1]+costoMO[j+1]) Entonces
                    // Intercambio manual
                    Definir auxC Como Cadena
                    Definir auxR Como Real
                    auxC <- codOrden[j]
                    codOrden[j] <- codOrden[j+1]
```

```
                                        codOrden[j+1]<-auxC
                                        auxC <- codEquipo[j]
                                        codEquipo[j] <- codEquipo[j+1]
                                        codEquipo[j+1]<-auxC
                                        auxC <- tecnico[j]
                                        tecnico[j] <- tecnico[j+1]
                                        tecnico[j+1]<-auxC
                                        auxC <- estado[j]
                                        estado[j] <- estado[j+1]
                                        estado[j+1]<-auxC
                                        auxR <- costoEst[j]
                                        costoEst[j] <- costoEst[j+1]
                                        costoEst[j+1]<-auxR
                                        auxR <- costoRep[j]
                                        costoRep[j] <- costoRep[j+1]
                                        costoRep[j+1]<-auxR
                                        auxR <- costoMO[j]
                                        costoMO[j] <- costoMO[j+1]
                                        costoMO[j+1]<-auxR
                                    FinSi
                                FinPara
                            FinPara
                            Escribir 'Ordenado por costo'
                    FinSegún
                Hasta Que opcion=0
            FinAlgoritmo
```

6) DIAGRAMA DE FLUJO



7) CODIGO EN C++

```
#include <iostream>
#include <string>
#include <iomanip>
#include <ctime>
#include <limits>
#include <vector>

using namespace std;

// --- FUNCIONES DE VALIDACIÓN ---
int leerEntero(string mensaje) {
```

```
    int valor;
    while (true) {
        cout << mensaje;
        if (cin >> valor && valor >= 0) return valor;
        cout << "[!] Error: Ingrese un numero entero valido.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

double leerDouble(string mensaje) {
    double valor;
    while (true) {
        cout << mensaje;
        if (cin >> valor && valor >= 0) return valor;
        cout << "[!] Error: Ingrese un valor numerico positivo.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

// --- CLASES ---
class Tecnico {
private:
    string codigo, nombre;
public:
    Tecnico(string c = "", string n = "") : codigo(c), nombre(n) {}
    string getCodigo() { return codigo; }
    string getNombre() { return nombre; }
};

class Mantenimiento {
private:
    string codigoOrden, codigoEquipo, tipo, tecnico, estado;
    int fechaReporte, fechaInicio, fechaCierre;
    double costoEstimado, costoRepuestos, costoManoObra;

public:
    Mantenimiento(string co, string ce, int fr, string t, string tec, double cEst)
        : codigoOrden(co), codigoEquipo(ce), fechaReporte(fr), tipo(t), tecnico(tec),
          costoEstimado(cEst), estado("Pendiente"), fechaInicio(0), fechaCierre(0),
          costoRepuestos(0), costoManoObra(0) {}
```

```
    // CORRECCIÓN DEL FRAGMENTO DEFECTUOSO (REQUISITO GRUPO D)
    int calcularDias() {
        if (fechaCierre == 0 || fechaCierre < fechaInicio) return 0;
        return fechaCierre - fechaInicio;
    }

    string getCodigo() { return codigoOrden; }
    string getTecnico() { return tecnico; }
    double getCostoTotal() { return costoEstimado + costoRepuestos + costoManoObra; }

    void finalizarOrden(int fCierre, double rep, double mano) {
        fechaInicio = fechaReporte + 1; // Simulación de inicio
        fechaCierre = fCierre;
        costoRepuestos = rep;
        costoManoObra = mano;
        estado = "Completado";
    }

    void mostrarFila() {
        cout << left << setw(10) << codigoOrden << setw(10) << codigoEquipo
            << setw(12) << estado << setw(12) << tecnico
            << "$" << fixed << setprecision(2) << setw(10) << getCostoTotal()
            << " Dias: " << calcularDias() << endl;
    }
};

class SistemaMantenimiento {
private:
    Mantenimiento* ordenes[100];
    int cont;
    string vocabulario[10] = {"FALLA", "DIAGNOSTICO", "REPUESTOS", "TECNICO", "HERRAMIENTA",
                              "MANTENIMIENTO", "REPARACION", "CIRCUITO", "VOLTAJE", "INSTITUCIONAL"};

    // RECURSIVIDAD ÚTIL PARA EL AHORCADO
    bool buscarLetraRecursiva(string palabra, char letra, int index) {
        if (index >= palabra.length()) return false;
        if (palabra[index] == letra) return true;
        return buscarLetraRecursiva(palabra, letra, index + 1);
    }

public:
```

```
SistemaMantenimiento() : cont(0) {}

void registrar() {
    if (cont >= 100) { cout << "Limite alcanzado.\n"; return; }
    string co, ce, t, tec;
    cout << "ID Orden: "; cin >> co;
    cout << "ID Equipo: "; cin >> ce;
    int fr = leerEntero("Fecha Reporte (AAAAMMDD): ");
    cout << "Tipo (P-Preventivo/C-Correctivo): "; cin >> t;
    cout << "ID Tecnico: "; cin >> tec;
    double cost = leerDouble("Costo Estimado: ");
    ordenes[cont++] = new Mantenimiento(co, ce, fr, t, tec, cost);
}

void completarOrden() {
    string id;
    cout << "Ingrese ID de Orden a completar: "; cin >> id;
    for (int i = 0; i < cont; i++) {
        if (ordenes[i]->getCodigo() == id) {
            int fc = leerEntero("Fecha Cierre (AAAAMMDD): ");
            double cr = leerDouble("Costo Repuestos: ");
            double cm = leerDouble("Costo Mano de Obra: ");
            ordenes[i]->finalizarOrden(fc, cr, cm);
            cout << "Orden actualizada.\n";
            return;
        }
    }
    cout << "Orden no encontrada.\n";
}

// ORDENAMIENTO MANUAL (MÉTODO BURBUJA - PROHIBIDO USAR SORT)
void ordenarPorCosto() {
    for (int i = 0; i < cont - 1; i++) {
        for (int j = 0; j < cont - i - 1; j++) {
            if (ordenes[j]->getCostoTotal() < ordenes[j+1]->getCostoTotal()) {
                Mantenimiento* temp = ordenes[j];
                ordenes[j] = ordenes[j+1];
                ordenes[j+1] = temp;
            }
        }
    }
    cout << "[!] Lista ordenada por Costo Total (Descendente).\n";
```

```
}

void mostrarReporte() {
    cout << "\n" << string(60, '-') << "\n";
    cout << left << setw(10) << "ORDEN" << setw(10) << "EQUIPO"
        << setw(12) << "ESTADO" << setw(12) << "TECNICO" << setw(11) << "COSTO" << "TIEMPO\n";
    for (int i = 0; i < cont; i++) ordenes[i]->mostrarFila();
    cout << string(60, '-') << "\n";
}

void jugarAhorcado() {
    string palabra = vocabulario[rand() % 10];
    string oculta(palabra.length(), '_');
    int intentos = 6;
    char l;

    cout << "\n--- JUEGO DE AHORCADO TECNICO ---\n";
    while (intentos > 0 && oculta != palabra) {
        cout << "\nPalabra: " << oculta << " | Intentos: " << intentos << "\nLetra: ";
        cin >> l; l = toupper(l);

        if (buscarLetraRecursiva(palabra, l, 0)) {
            for (int i = 0; i < palabra.length(); i++)
                if (palabra[i] == l) oculta[i] = l;
        } else {
            intentos--;
            cout << "Incorrecto!\n";
        }
    }
    if (oculta == palabra) cout << "\nFELICIDADES! Ganaste: " << palabra << endl;
    else cout << "\nPERDISTE. La palabra era: " << palabra << endl;
}

void menu() {
    int op;
    do {
        cout << "\nSISTEMA MANTENIMIENTO UTA - GRUPO D\n";
            cout << "1. Registrar Orden\n2. Completar Orden (Costos/Fechas)\n3. Reporte y Ordenar\n4. Jugar
Ahorcado\n0. Salir\nOpcion: ";
        if (!(cin >> op)) { cin.clear(); cin.ignore(1000, '\n'); continue; }
        switch(op) {
            case 1: registrar(); break;
```

```
            case 2: completarOrden(); break;
            case 3: ordenarPorCosto(); mostrarReporte(); break;
            case 4: jugarAhorcado(); break;
        }
    } while (op != 0);
  }
};

int main() {
  srand(time(NULL));
  SistemaMantenimiento sistema;
  sistema.menu();
  return 0;
}
```

8) PRUEBAS DE SOLUCIÓN

Primer caso

```
SISTEMA MANTENIMIENTO UTA - GRUPO D
1. Registrar Orden
2. Completar Orden (Costos/Fechas)
3. Reporte y Ordenar
4. Jugar Ahorcado
0. Salir
Opcion: 1
ID Orden: 5464
ID Equipo: 44488
Fecha Reporte (AAAAMMDD): 14/02/2026
Tipo (P-Preventivo/C-Correctivo): ID Tecnico: P
Costo Estimado: 45

SISTEMA MANTENIMIENTO UTA - GRUPO D
1. Registrar Orden
2. Completar Orden (Costos/Fechas)
3. Reporte y Ordenar
4. Jugar Ahorcado
0. Salir
Opcion: 2
Ingrese ID de Orden a completar: 5464
Fecha Cierre (AAAAMMDD): 24/05/2027
Costo Repuestos: [!] Error: Ingrese un valor numerico positivo.
Costo Repuestos: 14
Costo Mano de Obra: 89
Orden actualizada.

SISTEMA MANTENIMIENTO UTA - GRUPO D
1. Registrar Orden
2. Completar Orden (Costos/Fechas)
3. Reporte y Ordenar
4. Jugar Ahorcado
0. Salir
Opcion: 3
[!] Lista ordenada por Costo Total (Descendente).

------------------------------------------------------------
ORDEN     EQUIPO    ESTADO      TECNICO     COSTO      TIEMPO
5464      44488     Completado  P           $148.00     Dias: 9
------------------------------------------------------------

SISTEMA MANTENIMIENTO UTA - GRUPO D
1. Registrar Orden
2. Completar Orden (Costos/Fechas)
3. Reporte y Ordenar
4. Jugar Ahorcado
```

```
SISTEMA MANTENIMIENTO UTA - GRUPO D
1. Registrar Orden
2. Completar Orden (Costos/Fechas)
3. Reporte y Ordenar
4. Jugar Ahorcado
0. Salir
Opcion: 4

--- JUEGO DE AHORCADO TECNICO ---

Palabra: __________ | Intentos: 6
Letra: M
Incorrecto!

Palabra: __________ | Intentos: 5
Letra: P
Incorrecto!

Palabra: __________ | Intentos: 4
Letra: l
Incorrecto!

Palabra: __________ | Intentos: 3
Letra: I

Palabra: _I_____I__ | Intentos: 3
Letra: F
Incorrecto!

Palabra: _I_____I__ | Intentos: 2
Letra: C

Palabra: _I_____IC_ | Intentos: 2
Letra: R
Incorrecto!

Palabra: _I_____IC_ | Intentos: 1
Letra: O

Palabra: _I___O__ICO | Intentos: 1
Letra: A

Palabra: _IA__O__ICO | Intentos: 1
Letra: T

Palabra: _IA__O_TICO | Intentos: 1
Letra: D
```

```
Palabra: _I_____IC_ | Intentos: 2
Letra: R
Incorrecto!

Palabra: _I_____IC_ | Intentos: 1
Letra: O

Palabra: _I___O__ICO | Intentos: 1
Letra: A

Palabra: _IA__O__ICO | Intentos: 1
Letra: T

Palabra: _IA__O_TICO | Intentos: 1
Letra: D

Palabra: DIA__O_TICO | Intentos: 1
Letra: G

Palabra: DIAG_O_TICO | Intentos: 1
Letra: N

Palabra: DIAGNO_TICO | Intentos: 1
Letra: S

FELICIDADES! Ganaste: DIAGNOSTICO

SISTEMA MANTENIMIENTO UTA - GRUPO D
1. Registrar Orden
2. Completar Orden (Costos/Fechas)
3. Reporte y Ordenar
4. Jugar Ahorcado
0. Salir
Opcion: 0
```

Segundo caso

```
SISTEMA MANTENIMIENTO UTA - GRUPO D
1. Registrar Orden
2. Completar Orden (Costos/Fechas)
3. Reporte y Ordenar
4. Jugar Ahorcado
0. Salir
Opcion: 1
ID Orden: 64
ID Equipo: 654
Fecha Reporte (AAAAMMDD): 24/8/2026
Tipo (P-Preventivo/C-Correctivo): ID Tecnico: C
Costo Estimado: 65

SISTEMA MANTENIMIENTO UTA - GRUPO D
1. Registrar Orden
2. Completar Orden (Costos/Fechas)
3. Reporte y Ordenar
4. Jugar Ahorcado
0. Salir
Opcion: 3
[!] Lista ordenada por Costo Total (Descendente).

------------------------------------------------------------
ORDEN     EQUIPO    ESTADO      TECNICO     COSTO      TIEMPO
64        654       Pendiente   C           $65.00      Dias: 0
------------------------------------------------------------

SISTEMA MANTENIMIENTO UTA - GRUPO D
1. Registrar Orden
2. Completar Orden (Costos/Fechas)
3. Reporte y Ordenar
4. Jugar Ahorcado
0. Salir
Opcion: 4

--- JUEGO DE AHORCADO TECNICO ---

Palabra: _______ | Intentos: 6
Letra: A
Incorrecto!

Palabra: _______ | Intentos: 5
Letra: I

Palabra: ____I__ | Intentos: 5
Letra: O
```

```
Palabra: ____I_O | Intentos: 5
Letra: T

Palabra: T___I_O | Intentos: 5
Letra: N

Palabra: T__NI_O | Intentos: 5
Letra: C

Palabra: T_CNICO | Intentos: 5
Letra: E

FELICIDADES! Ganaste: TECNICO

SISTEMA MANTENIMIENTO UTA - GRUPO D
1. Registrar Orden
2. Completar Orden (Costos/Fechas)
3. Reporte y Ordenar
4. Jugar Ahorcado
0. Salir
Opcion: 0

Process returned 0 (0x0)   execution time : 81.517 s
Press any key to continue.
```

9) JUSTIFICACIÓN DEL FRAGMENTO

```
int diasProceso(int inicio, int cierre){
        int dias;
        if(cierre < inicio)
        dias = inicio - cierre;
        else
        dias = cierre + inicio;
        return dias;
}
```

En este fragmento podemos observar las partes en las cuales este presenta errores.

Si la fecha de cierre es menor que la de inicio, debería devolver un valor inválido (ej. `-1`) o indicar error, no calcular inicio - cierre.

En el caso normal, el cálculo debería ser cierre - inicio, no cierre + inicio.

No se valida que las fechas sean positivas o coherentes.

El código puede devolver resultados absurdos (sumar fechas).

La función no explica qué ocurre si los datos son inválidos.

DEPURACIÓN Y CORRECCIÓN DEL CÓDIGO

```
int diasProceso(int inicio, int cierre){
```

```
    int dias;

    if(cierre >= inicio)
        dias = cierre - inicio;
    else
        dias = 0; // o manejar error

    return dias;
}
```

En el código original, la duración del proceso se obtenía sumando las fechas de inicio y cierre, lo cual carece de sentido lógico, ya que las fechas representan momentos en el tiempo y no deben combinarse mediante una suma.

La corrección consistió en reemplazar esa operación por una resta (cierre - inicio), que refleja de manera adecuada el número de días transcurridos entre ambos momentos.

Además, se incorporó una validación para garantizar que la fecha de cierre no sea anterior a la de inicio. De esta forma se evitan resultados negativos y se asegura la coherencia de los datos ingresados.

Con estas modificaciones, la función ahora devuelve un cálculo correcto y confiable de la duración del proceso, y en caso de datos inválidos, se evita generar resultados erróneos.

2.8 Habilidades blandas empleadas en la práctica

☒ Liderazgo
☒ Trabajo en equipo
☒ Comunicación asertiva
☐ La empatía
☒ Pensamiento crítico
☐ Flexibilidad
☐ La resolución de conflictos
☐ Adaptabilidad
☒ Responsabilidad

2.9 Conclusiones

El desarrollo del sistema de mantenimiento técnico permitió aplicar de manera práctica los conocimientos adquiridos en programación en visual code studio - C + +, especialmente en el uso de estructuras de control,

funciones y manejo de datos. A través de este proyecto se logró implementar un sistema capaz de registrar, procesar y mostrar información relevante sobre órdenes de mantenimiento.

El sistema facilita la organización de datos como costos, tiempos de reparación y técnicos responsables, permitiendo obtener reportes útiles para el análisis del estado de los equipos. Además, se integró el módulo Ahorcado, el cual se adaptó al contexto del proyecto, demostrando la capacidad de incorporar funcionalidades adicionales dentro de un mismo sistema.

En general, el proyecto permitió desarrollar habilidades de análisis, diseño y programación, cumpliendo con los objetivos planteados en el examen.

2.10 Recomendaciones

Se recomienda mejorar el sistema incorporando validaciones más estrictas para evitar errores en el ingreso de datos, especialmente en fechas y costos. Asimismo, sería útil ampliar las funcionalidades del sistema, agregando más opciones de búsqueda y reportes detallados.

También se sugiere mejorar el módulo Ahorcado incluyendo más palabras, niveles de dificultad y un sistema de puntuación más completo. De igual manera, se podría optimizar el código para manejar una mayor cantidad de registros de forma eficiente.

Finalmente, se recomienda seguir utilizando herramientas de control de versiones para mantener un mejor control del desarrollo del proyecto y facilitar futuras mejoras.

2.11 Referencias bibliográficas

Programación ATS. (2016, May 1). *40. Programación en C++ || Cadenas || Introducción a las cadenas de caracteres* [Video]. YouTube. https://www.youtube.com/watch?v=D7hbaJhgzBs